0.68. Voir la position du A en direct

Le 0.67, plus deux lignes dans la boucle : nombre(0, 17, x) et nombre(4, 17, y). La position du A s'affiche à chaque image, pendant qu'on le déplace.

1. Ce qu'il faut comprendre

C'est le 0.67, plus deux lignes dans la boucle : `nombre(0, 17, x);` et `nombre(4, 17, y);`.

Ce qui est nouveau ici : `nombre()` DANS la boucle. Au 0.20, `nombre()` était appelé une fois, après un trajet : il montrait la valeur à ce moment-là. Ici, il est dans `while (true)` : il est rappelé à **chaque image**, 60 fois par seconde. L'écran montre donc toujours la valeur **du moment** : c'est de l'affichage **en temps réel**.

Ce qu'on voit : en bas à gauche, trois chiffres pour `x` (la colonne du A), et à côté, trois chiffres pour `y` (sa ligne). Appuie sur les flèches : les nombres changent en même temps que la lettre bouge. Au départ, `009` et `008`, le milieu de l'écran.

Pourquoi avant `poser` : si le A va sur la ligne 17, il passe **par-dessus** les nombres, parce qu'il est posé après eux. Les nombres sont réécrits à l'image suivante, dès que le A repart.

Sans rien écrire, la page le montre aussi : sous la console, ouvre « 🔬 Inspecteur », onglet **Variables** : `x`, `y` et `attente` y sont, par leur nom, mis à jour 4 fois par seconde.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <effacer>` → 0.16.3. La fonction effacer() — effacer des cases
> - `#include <bouton>` → 0.65.5. La fonction bouton() — lire la manette
> - `#include <nombre>` → 0.8.3. La fonction nombre() — écrire un nombre
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **bouton()** — Une fonction de la console qui dit si un bouton de la manette est enfoncé : 1 oui, 0 non. Les boutons : A, B, HAUT, BAS, GAUCHE, DROITE, START, SELECT.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **if (si)** — Teste une condition : si elle est vraie, on fait le bloc qui suit.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **opérateurs logiques** — && « et » (les deux doivent être vrais), || « ou » (un seul suffit), ! « pas ».
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// x, y, attente : voir l'onglet variables.h. (#include veut être seul sur sa ligne.)`	Un commentaire : une phrase pour celui qui lit (« x, y, attente : voir l'onglet variables.h. (#include veut être seul sur sa ligne.) »). La console l'ignore.
`#include <effacer>    // efface des cases, ou tout le fond`	Demande à la console « effacer » (ce que c'est : efface des cases, ou tout le fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « efface des cases, ou tout le fond ».)
`#include <bouton>     // lit un bouton de la manette`	Demande à la console « bouton » (ce que c'est : lit un bouton de la manette). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « lit un bouton de la manette ».)
`#include <nombre>     // écrit un nombre en chiffres`	Demande à la console « nombre » (ce que c'est : écrit un nombre en chiffres). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un nombre en chiffres ».)
`#include <poser>      // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include "variables.h"`	Verse ici le fichier « variables.h » (un autre onglet du programme) : tout se passe comme si son texte était écrit à cet endroit.
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().
`image(); // attend l'image suivante`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « attend l'image suivante ».)
`if (attente > 0) attente--;`	Une décision : si attente est plus grand que 0 alors on fait « attente-- ». Sinon, on passe.
`if (attente == 0) {`	Une décision : si attente est égal à 0, on fait ce qui est entre les accolades. Sinon, on passe.
`effacer(x, y, 1); // efface l'ancienne place`	Appel de la fonction effacer de la console, avec les arguments x, y, 1. Ce qu'elle fait : efface un texte à l'écran : « quoi » est le texte lui-même, ou un nombre de cases. Avec un Mot : effacer(SALUT). (Le commentaire le dit : « efface l'ancienne place ».)
`if (bouton(DROITE) && x < 19) { x++; attente = 8; }`	Une décision : si le bouton DROITE est enfoncé et x est plus petit que 19 alors on fait « x++; attente = 8; } ». Sinon, on passe.
`if (bouton(GAUCHE) && x > 0)  { x--; attente = 8; }`	Une décision : si le bouton GAUCHE est enfoncé et x est plus grand que 0 alors on fait « x--; attente = 8; } ». Sinon, on passe.
`if (bouton(BAS) && y < 17)    { y++; attente = 8; }`	Une décision : si le bouton BAS est enfoncé et y est plus petit que 17 alors on fait « y++; attente = 8; } ». Sinon, on passe.
`if (bouton(HAUT) && y > 0)    { y--; attente = 8; }`	Une décision : si le bouton HAUT est enfoncé et y est plus grand que 0 alors on fait « y--; attente = 8; } ». Sinon, on passe.

`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`// Les deux lignes ajoutées : la position du A, réécrite à CHAQUE image.`	Un commentaire : une phrase pour celui qui lit (« Les deux lignes ajoutées : la position du A, réécrite à CHAQUE image. »). La console l'ignore.
`//   nombre(0, 17, x);        nombre(4, 17, y);`	Un commentaire : une phrase pour celui qui lit (« nombre(0, 17, x); nombre(4, 17, y); »). La console l'ignore.
`//          |  |   |                |  |   |`	Un commentaire : une phrase pour celui qui lit (« |||||| »). La console l'ignore.
`//          |  |   +-- x : la colonne |  |   +-- y : la ligne du A`	Un commentaire : une phrase pour celui qui lit (« || +-- x : la colonne || +-- y : la ligne du A »). La console l'ignore.
`//          |  +------ ligne 17 : tout en bas de l'écran`	Un commentaire : une phrase pour celui qui lit (« | +------ ligne 17 : tout en bas de l'écran »). La console l'ignore.
`//          +--------- colonne 0 et colonne 4 : côte à côte`	Un commentaire : une phrase pour celui qui lit (« +--------- colonne 0 et colonne 4 : côte à côte »). La console l'ignore.
`//   En bas de l'écran :  009 008   ← le A au départ, en (9, 8)`	Un commentaire : une phrase pour celui qui lit (« En bas de l'écran : 009 008 ← le A au départ, en (9, 8) »). La console l'ignore.
`//                        012 010   ← après 3 pas à droite et 2 en bas`	Un commentaire : une phrase pour celui qui lit (« 012 010 ← après 3 pas à droite et 2 en bas »). La console l'ignore.
`nombre(0, 17, x);` `// la colonne, en direct`	Appel de la fonction nombre de la console, avec les arguments 0, 17, x. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la colonne, en direct ».)
`nombre(4, 17, y);` `// la ligne, en direct`	Appel de la fonction nombre de la console, avec les arguments 4, 17, y. Ce qu'elle fait : écrit un nombre calculé, en base dix. Le 4e argument dit combien de chiffres (3 par défaut). Un 5e est la TAILLE, sans limite : nombre(2, 2, score, 3, 4) écrit le score quatre fois plus grand. (Le commentaire le dit : « la ligne, en direct ».)
`poser(x, y, ALPHABET[0]);` `// la lettre, par-dessus tout`	Appel de la fonction poser de la console, avec les arguments x, y, ALPHABET[0]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair). (Le commentaire le dit : « la lettre, par-dessus tout ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **615 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        60 o  ← la console, à l'allumage  — la mise en route de la console
  main()          169 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
```

```
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces    12 o  ← EffacerCases, EcrireNombre  — attend de pouvoir écrire une case
  AdresseCase      10 o  ← effacer(), poser()  — trouve l'adresse d'une case de l'écran
  EffacerCases     12 o  ← effacer()  — efface des cases
  EcrireNombre     33 o  ← nombre()  — écrit un nombre en chiffres
  CopierTuiles     40 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  LireManette      37 o  ← bouton()  — lit les huit boutons
  Tuiles          176 o  ← CopierTuiles  — 11 lettres sur 43
Lettres gravées : A0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche,
DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes
Fonctions du langage appelées :
  bouton()  × 4    20 o sur place, dans main()  + LireManette (37 o)
  nombre()  × 2    20 o sur place, dans main()  + EcrireNombre (33 o)
  effacer() × 1    15 o sur place, dans main()  + AdresseCase (10 o), EffacerCases (12 o)
  poser()   × 1    13 o sur place, dans main()  + AdresseCase (10 o)
  image()   × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// x, y, attente : voir l'onglet variables.h. (#include veut être seul sur sa ligne.)

#include <effacer>    // efface des cases, ou tout le fond
#include <bouton>     // lit un bouton de la manette
#include <nombre>     // écrit un nombre en chiffres
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  while (true) {
    image();                                        // attend l'image suivante

    if (attente > 0) attente--;

    if (attente == 0) {
      effacer(x, y, 1);                             // efface l'ancienne place
      if (bouton(DROITE) && x < 19) { x++; attente = 8; }
      if (bouton(GAUCHE) && x > 0)  { x--; attente = 8; }
      if (bouton(BAS) && y < 17)    { y++; attente = 8; }
      if (bouton(HAUT) && y > 0)    { y--; attente = 8; }
    }

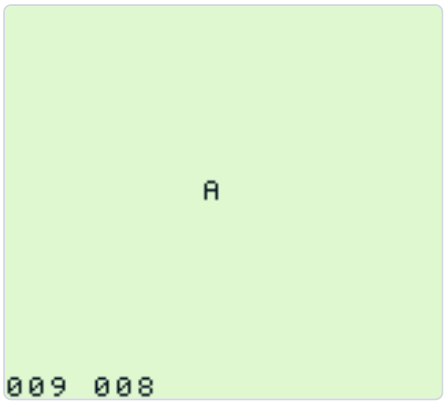
    // Les deux lignes ajoutées : la position du A, réécrite à CHAQUE image.
    //
    //   nombre(0, 17, x);        nombre(4, 17, y);
    //          |  |   |                |  |   |
    //          |  |   +-- x : la colonne |  |   +-- y : la ligne du A
    //          |  +------ ligne 17 : tout en bas de l'écran
    //          +--------- colonne 0 et colonne 4 : côte à côte
    //
    //   En bas de l'écran :  009 008   ← le A au départ, en (9, 8)
    //                        012 010   ← après 3 pas à droite et 2 en bas
    nombre(0, 17, x);                               // la colonne, en direct
    nombre(4, 17, y);                               // la ligne, en direct

    poser(x, y, ALPHABET[0]);                       // la lettre, par-dessus tout
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

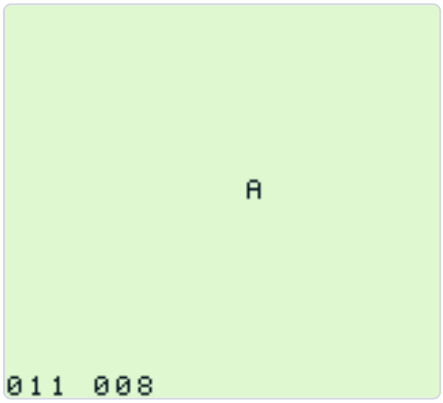
la toute première image — la console vient de démarrer — image 1



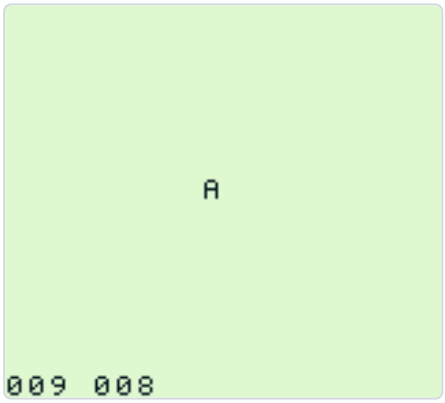
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

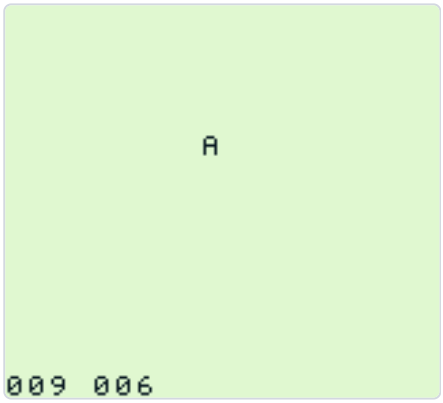
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



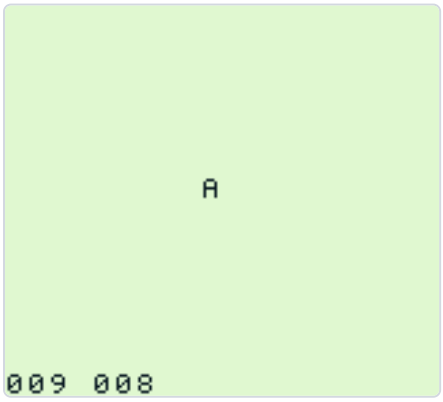
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

> Le A qui se déplace avec les flèches ; en bas à gauche, sa colonne et sa ligne changent en même temps.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, 009 et 008 (« 009 008 »)
- ✓ le A a bougé (x = 13, y = 5)
- ✓ l'écran montre sa colonne (« 013 »)
- ✓ et sa ligne (« 005 »)

Ensuite : **0.68.1. Voir la position du A en direct — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++